2026 S.T. Yau High School Science Award (Asia)

Research Report

Saddle Escape Efficiency: A Novel Metric and Its Criterion-Dependence in Non-Convex Landscapes

Registration Number:	Comp - 101
Team Member:	Arnav Dhiman
School:	Greenfingers Global School
City:	Navi Mumbai
Country:	India
Supervising Teacher:	Nilima Gadgil
Job Title:	Head of Department, Mathematics
School:	Greenfingers Global School
City:	Navi Mumbai
Country:	India

17 August 2026

Author: Arnav Dhiman

Abstract

Gradient descent and its adaptive variants are the backbone of modern machine learning, but non-convex loss landscapes contain saddle points that slow down or stall optimisation. While extensive theoretical work has established that adaptive methods escape saddle points [8, 6, 7], there is no metric to directly quantify and compare the efficiency of different optimisers in this regard in practice. The selection of the appropriate optimiser for a saddle-heavy problem is still mostly intuition-based [16].

We propose a new metric, Saddle Escape Efficiency (SEE = $P_{\text{esc}}/\bar{\tau}$), which combines escape probability and mean escape time, and use it to reveal a fundamental issue in saddle-escape benchmarking: the choice of escape criterion, the rule used to declare escape, is not a trivial implementation detail, but can flip optimiser rankings completely. Four independent escape criteria are tested on identical trajectories of six optimisers and seven verified 2D benchmark functions. Distance and curvature criteria yield Spearman rank correlations as low as $\rho = -0.81$ on the same data. The agreement of two criteria based on independent geometric principles on five of seven functions mutually validates that they capture a real underlying escape event, and that $\rho = +1.00$. The analysis is extended to 10-dimensional and 50-dimensional generalisations of Rastrigin and Ackley, where the distance criterion saturates to 1.0 for every optimiser while the curvature criterion still discriminates, and to saddle points found in the parameter space of a small neural network.

Thus any saddle-escape metric must state and justify its criterion explicitly. A value given under a distance criterion cannot be compared to a value given under a curvature criterion since the two can identify totally different optimisers as best for the same problem. We argue that geometry-grounded criteria should be the default for future saddle-escape benchmarking.

Acknowledgements

I thank Nilima Gadgil for reviewing drafts of this work and for guidance on the mathematical framing. Experiments were run on Kaggle's T4 GPU environment using PyTorch, NumPy, SciPy, and Matplotlib.

Claude (Anthropic) was used for grammatical feedback and to improve the flow of writing. No part of the research, analysis, code, or scientific content was generated by AI. All code, experiments, and findings are my own.

All Code can be found at https://github.com/arnavd371/Saddle-Escape-Efficiency

Contents

Abstract **i**

Acknowledgements **ii**

1 Introduction **1**
1.1 Background 1
1.2 Research Gap and Contributions 1

2 Related Work **1**

3 Mathematical Background **3**
3.1 Gradient Descent 3
3.2 Saddle Points 3

4 Methodology **3**
4.1 Saddle Escape Efficiency 3
4.2 Saddle Finding 4
4.3 Benchmark Functions 4
4.4 Optimisers and Setup 4
4.5 The Four Escape Criteria 5

5 Results **5**
5.1 Verified Saddle Points 5
5.2 Best-Learning-Rate SEE 6
5.3 Criterion-Dependence 7
5.4 Higher Dimensions 10
5.5 Neural Network Loss Surface 11

6 Limitations and Future Work **12**

7 Conclusion **12**

1. Introduction

1.1. Background

Gradient descent is one of the most fundamental algorithms in the field of machine learning. It works by moving in the direction of the negative slope also called gradient, with step size which is based on a learning rate η_k. In convex settings, it converges most of the times, but in non-convex settings it gets stuck at saddle points: points where the gradient or slope is zero but the hessian has both positive and negative eigenvalues. Close to a saddle point, the gradient or slope is small in all directions and the optimizer stalls and the algorithm becomes stuck. This is often a big problem when implementing gradient descent algorithm or any gradient based algorithm.

Dauphin et al. [1] showed in their work that in the high-dimensional loss landscapes of deep networks, saddle points are a very big obstacle, not only local minima. This makes saddle escape behavior an essential property for any optimizer. To deal with non-convex landscapes, adaptive techniques such as AdaGrad [2], RMSProp [3], and Adam [4] have been proposed, and theoretical work has shown and confirmed that they are able to escape saddle points [8, 6, 7]. However, the choice of the optimizer to deal with a saddle-heavy problem is still mainly intuition-based [16] since there is no metric to directly measure saddle-escape efficiency.

1.2. Research Gap and Contributions

There is no single metric for how well an optimizer escapes saddle points. Current metrics like final loss or steps to convergence average over the entire trajectory, and say nothing about the saddle dynamics of the landscape. A very intuitive way to counter this is to declare escape when the iterate moves a fixed distance from the saddle, but this paper shows that criterion measures step size, not curvature change, and can produce rankings that are the direct opposite of geometry based criteria.

In this paper we introduce Saddle Escape Efficiency (SEE), define four independent escape criteria, and show that the choice of criterion is not a minor detail, but can completely reverse the rankings of optimizers depending on what you define as the criterion. The analysis is extended to six optimizers, seven benchmark functions, higher dimensions and a Neural Network.

2. Related Work

In high-dimensional non-convex problems: saddle points are the main challenge in optimisation as identified by Dauphin et al. [1] who used random matrix theory to show

that critical points on deep networks are dominated by saddle points instead of local minima, and proposed a saddle-free Newton method with modified Hessian eigenvalues to escape them straight. Ge et al. [12] first reported escape in polynomial time guarantee for stochastic gradient descent in strict saddle property, and Lee et al. [13] showed that by the Stable Manifold Theorem that random initialisation, converges to local gradient descent almost surely minimisers instead of saddle points.

For adaptive optimisers, Jin et al. [8] showed that perturbed gradient descent finds second-order stationary points in essentially dimension-free time. Staib et al. [6] gave the first second-order convergence guarantee for any adaptive method, showing that the preconditioner of RMSProp and Adam isotropizes gradient noise around stationary points, enabling faster saddle escape than SGD. Antonakopoulos et al. [7] proved that AdaGrad escapes strict saddle points from almost any initialisation without additional noise. These results show that adaptive methods leave saddle points but use iteration complexity as a metric, not a unified efficiency metric that brings probability and time together.

Further context is provided by work on loss landscapes. Goodfellow et al. [17] found that there are no significant loss barriers along linear interpolation paths of trained networks, implying that saddle escape rather than local minima is the main challenge. Li et al. [18] introduced filter normalisation for landscape visualisation and found that skip connections result in flatter minima and that Hessian sharpness correlates with generalisation. Large-batch training leads to sharp minima with large Hessian eigenvalues, directly relating curvature to optimiser behaviour, according to Keskar et al. [19]. Foret et al. [20] proposed Sharpness-Aware Minimisation, which explicitly aims for flat minima by minimising loss over neighbourhoods of parameters, thus further motivating curvature-based analysis of optimiser dynamics.

However, there is no paper in the literature that defines a metric of how efficiently different optimisers escape saddle points, and none that investigates whether the rankings of optimisers change depending on the escape criterion used. Benchmarking papers compare optimisers on final accuracy or steps to convergence [21] which average over the full trajectory and say nothing about saddle dynamics. This paper introduces SEE to fill the first gap, and shows rank inversions as strong as $\rho = -0.81$ between distance and curvature criteria to fill the second.

3. Mathematical Background

3.1. Gradient Descent

Given $f : \mathbb{R}^n \to \mathbb{R}$, gradient descent follows the update rule as

$$x_{k+1} = x_k - \eta_k \nabla f(x_k), \tag{1}$$

where $\eta_k > 0$ is the learning rate [9]. Methods that adapt or adaptive methods scale each coordinate's update as a function of the past gradients. AdaGrad divides by accumulated squared gradients [2]. RMSProp uses an exponential moving average to prevent the learning rate decaying to zero [3]. Adam adds momentum to RMSProp [4]. AdamW decouples weight decay from the gradient update [5].

3.2. Saddle Points

A point $x^* \in \mathbb{R}^n$ is a saddle point if the $\nabla f(x^*) = 0$ and the $H(x^*) = \nabla^2 f(x^*)$ has both positive and negative eigenvalues [10]. The direction v corresponding to the most negative eigenvalue $\lambda_{\min}$ is the natural escape direction. The curvature radius is given by

$$r_{\text{curv}} = \frac{1}{\sqrt{|\lambda_{\min}(H(x^*))|}}$$

is the natural length scale of the saddle's unstable manifold. A fixed distance criterion $\|x - x_s\| > r$ is on the wrong scale when $|\lambda_{\min}|$ varies by orders of magnitude across different functions.

4. Methodology

4.1. Saddle Escape Efficiency

Let P_{esc} be the empirical escape probability within $T_{\max}$ steps, and $\bar{\tau}$ the mean first escape step among all the successful trials.

$$\text{SEE} = \frac{P_{\text{esc}}}{\bar{\tau}}. \tag{2}$$

High SEE means the optimiser escapes reliably and quickly. The denominator penalises slow escape even when it is eventually guaranteed. Bootstrap 95% confidence intervals use 2000 resamples, reported as asymmetric percentile intervals [11].

4.2. Saddle Finding

For 2D functions a 240×240 grid is used and evaluated and the 800 candidates with smallest gradient norm are refined using `scipy.optimize.fsolve`. A point is accepted only if

$$\|\nabla f(x)\| < 10^{-6}, \quad \lambda_{\min}(H) < -10^{-4}, \quad \lambda_{\max}(H) > 10^{-4}.$$

Candidates are sorted by gradient norm then lexicographically, making the procedure fully deterministic. Up to three saddles per function are kept. Trials initialise at $x_0 \sim \mathcal{N}(x_s, 0.1^2 I)$.

For $d \geq 10$, 5000 random initialisations replace the grid, with the same acceptance criteria and Lanczos eigenvalue computation.

The escape eigenvector v for criterion C is the column of `torch.linalg.eigh`$(H(x_s))$ for $\lambda_{\min}$, normalised to unit length. No closed-form shortcut is used.

4.3. Benchmark Functions

Seven 2D functions are used. Rosenbrock is excluded because it has no saddle point. Booth yielded no verified saddle and is excluded.

1. Himmelblau: $f = (x^2 + y - 11)^2 + (x + y^2 - 7)^2$
2. Ackley: $f = -20e^{-0.2\sqrt{\frac{1}{n}\sum x_i^2}} - e^{\frac{1}{n}\sum \cos(2\pi x_i)} + 20 + e$
3. Rastrigin: $f = 10n + \sum_i (x_i^2 - 10\cos 2\pi x_i)$
4. Styblinski-Tang: $f = \frac{1}{2}\sum_i (x_i^4 - 16x_i^2 + 5x_i)$
5. Levy: $f = \sin^2(\pi w_1) + \sum_{i=1}^{n-1}(w_i - 1)^2[1 + 10\sin^2(\pi w_{i+1})] + (w_n - 1)^2[1 + \sin^2(2\pi w_n)]$, where $w_i = 1 + (x_i - 1)/4$
6. Beale: $f = (1.5 - x + xy)^2 + (2.25 - x + xy^2)^2 + (2.625 - x + xy^3)^2$
7. Schwefel: $f = 418.9829n - \sum_i x_i \sin\left(\sqrt{|x_i|}\right)$

4.4. Optimisers and Setup

Six optimisers: Gradient Descent (GD) (fixed rate), Adam ($\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$), RMSProp ($\alpha = 0.99$, $\varepsilon = 10^{-8}$), AdaGrad ($\varepsilon = 10^{-8}$), AdamW (weight_decay= 0.01), Stochastic Gradient Descent(SGD) (momentum= 0.9).
Learning rates: $\{0.001, 0.01, 0.05, 0.1, 0.2, 0.5\}$. $N = 200$ trials, $T_{\max} = 200$, `seed=42`.

4.5. The Four Escape Criteria

All four criteria run on the same trajectories, so any ranking difference is due to the criterion alone.

A (fixed radius): $\|x_t - x_s\| > r$, $r \in \{1.5, 2.0, 3.0\}$. Simple, no Hessian needed, but measures displacement not curvature.

B (curvature exit): $\lambda_{\min}(H(x_t)) > -\varepsilon$, $\varepsilon \in \{10^{-2}, 10^{-3}, 10^{-4}\}$. Tests whether the iterate has left the region of indefinite curvature.

C (eigenvector projection): $|\langle x_t - x_s, v\rangle| > c\, r_{\text{curv}}$, $c \in \{0.5, 1.0, 2.0\}$. Tests movement along the natural escape direction, scaled by r_{curv} so the threshold is comparable across functions.

D (loss drop): $f(x_t) < f(x_s) - \delta$ and $\lambda_{\min}(H(x_t)) > -10^{-3}$, $\delta \in \{0.25, 0.5, 1.0\}$. Requires both a function value decrease and departure from indefinite curvature.

Headline parameters: $r = 2.0$, $\varepsilon = 10^{-3}$, $c = 1.0$, $\delta = 0.5$.

5. Results

5.1. Verified Saddle Points

Table 1: Verified saddle points. $r_{\text{curv}} = 1/\sqrt{|\lambda_{\min}|}$.

Function	x_s	$\lambda_{\min}$	r_{curv}
Himmelblau	$(-0.128,\ -1.954)$	-50.61	0.141
Ackley	$(-0.577,\ \ 0.875)$	-17.11	0.242
Rastrigin	$(-0.503,\ \ 0.000)$	-392.73	0.050
Styblinski	$(\ 0.157,\ \ 2.747)$	-15.85	0.251
Levy	$(\ 1.000,\ -2.257)$	-2.96	0.582
Beale	$(\ 0.000,\ \ 1.000)$	-27.75	0.190
Schwefel	$(-5.239,\ \ 5.239)$	-0.40	1.574

5.2. Best-Learning-Rate SEE

Table 2: Best-LR SEE at headline parameters. Bold marks the highest per row per criterion.

Function	Optimiser	A	B	C	D
Himmelblau	GD	0.637	0.543	0.943	0.227
	Adam	0.216	0.143	1.000	0.143
	RMSProp	1.000	1.000	1.000	0.451
	AdaGrad	0.210	0.199	1.000	0.199
	AdamW	0.213	0.140	1.000	0.140
	SGD+mom	0.639	0.542	0.943	0.207
Ackley	GD	0.305	0.254	0.758	0.251
	Adam	0.000	0.342	1.000	0.333
	RMSProp	1.000	0.286	1.000	0.286
	AdaGrad	0.000	0.437	1.000	0.426
	AdamW	0.001	0.338	1.000	0.329
	SGD+mom	0.359	0.186	0.755	0.181
Rastrigin	GD	0.985	0.303	0.990	0.303
	Adam	0.000	0.709	1.000	0.709
	RMSProp	1.000	0.421	1.000	0.421
	AdaGrad	0.000	0.709	1.000	0.709
	AdamW	0.000	0.709	1.000	0.709
	SGD+mom	0.985	0.299	0.990	0.239
Styblinski	GD	0.549	0.444	0.749	0.221
	Adam	0.208	0.252	1.000	0.252
	RMSProp	1.000	1.000	1.000	0.500
	AdaGrad	0.200	0.267	1.000	0.267
	AdamW	0.207	0.249	1.000	0.249
	SGD+mom	0.548	0.442	0.749	0.230
Levy	GD	0.017	0.332	0.291	0.147
	Adam	0.103	0.746	0.610	0.255
	RMSProp	1.000	0.746	1.000	0.255
	AdaGrad	0.013	0.746	0.610	0.255
	AdamW	0.104	0.763	0.613	0.255
	SGD+mom	0.112	0.366	0.330	0.160
Beale	GD	0.585	0.230	0.897	0.228
	Adam	0.028	0.208	0.377	0.208
	RMSProp	1.000	0.213	0.350	0.213
	AdaGrad	0.037	0.212	0.372	0.212
	AdamW	0.027	0.216	0.372	0.216
	SGD+mom	0.587	0.244	0.897	0.244
Schwefel	GD	0.047	0.035	0.050	0.035
	Adam	0.207	0.072	0.256	0.072
	RMSProp	1.000	0.565	1.000	0.500
	AdaGrad	0.200	0.029	0.258	0.029
	AdamW	0.207	0.068	0.254	0.068
	SGD+mom	0.092	0.073	0.097	0.073

5.3. Criterion-Dependence

Table 3: Pairwise Spearman ρ between criteria ($n = 6$ optimisers).

Function	A-B	A-C	A-D	B-C	B-D	C-D
Himmelblau	+0.77	−0.41	+0.77	−0.41	+1.00	−0.41
Ackley	−0.81	−0.45	−0.81	+0.85	+1.00	+0.85
Rastrigin	−0.79	−0.45	−0.79	+0.88	+1.00	+0.88
Styblinski	+0.83	−0.41	−0.09	−0.41	+0.09	+0.83
Levy	+0.15	+0.49	+0.10	+0.83	+0.90	+0.86
Beale	+0.37	0.00	+0.37	+0.53	+1.00	+0.53
Schwefel	+0.49	+0.77	+0.49	+0.26	+1.00	+0.26

Table 4: Kendall's W across A, B, C, D, ordered by $|\lambda_{\min}|$.

Function	W	$\lvert\lambda_{\min}\rvert$
Schwefel	0.657	0.40
Levy	0.588	2.96
Beale	0.593	27.75
Styblinski	0.334	15.85
Ackley	0.288	17.11
Himmelblau	0.420	50.61
Rastrigin	0.275	392.73

Three findings come out of these tables.

Finding 1: Within each criterion family, rankings are stable. Spearman ρ between extreme parameter values averages above 0.90 for A, B, and D. Criterion B is $\rho = 1.00$ on every function. The disagreement between criteria is structural and not a tuning artefact.

Finding 2: Distance and curvature criteria can invert rankings. On Ackley and Rastrigin, A-B $\rho = -0.81$ and -0.79. Criterion A puts RMSProp first on every function. Criterion B puts Adam, AdaGrad, and AdamW jointly first on Rastrigin at SEE = 0.709, with RMSProp third at 0.421. This reversal holds across all six learning rates.

The reason is straightforward. RMSProp divides each coordinate's update by the square root of a running average of squared gradients. Near a saddle the gradient is small, the denominator is also small, and this produces a large normalised step that crosses any fixed-radius boundary quickly regardless of whether the Hessian has changed sign. Criterion A measures step size. Criterion B measures whether the curvature has actually changed.

Finding 3: Criteria B and D agree near-perfectly. B-D $\rho = +1.00$ on five of seven functions. Two criteria derived from independent principles one based on the sign of $\lambda_{\min}(H(x_t))$, the other on function value plus a curvature check produce identical optimiser rankings on most functions. This confirms they are measuring the same real event.

The functions where A and B agree ($\rho > 0$) have flat saddles ($|\lambda_{\min}| < 52$). Ackley and Rastrigin, where the inversion occurs, have sharp curvature and oscillatory cosine terms. Criterion C saturates near 1.0 on sharp saddles because the gradient pushes any iterate along the escape eigenvector regardless of step scaling. On flat saddles (Levy, Beale, Schwefel) criterion C discriminates, ranging from 0.050 to 0.897.

The Spearman correlation between $|\lambda_{\min}|$ and Kendall's W across the original five functions is $\rho = -0.70$ (exact permutation $p = 0.233$, $n = 5$). The direction is consistent with sharper saddles producing less criterion agreement, but it is not significant at $n = 5$.

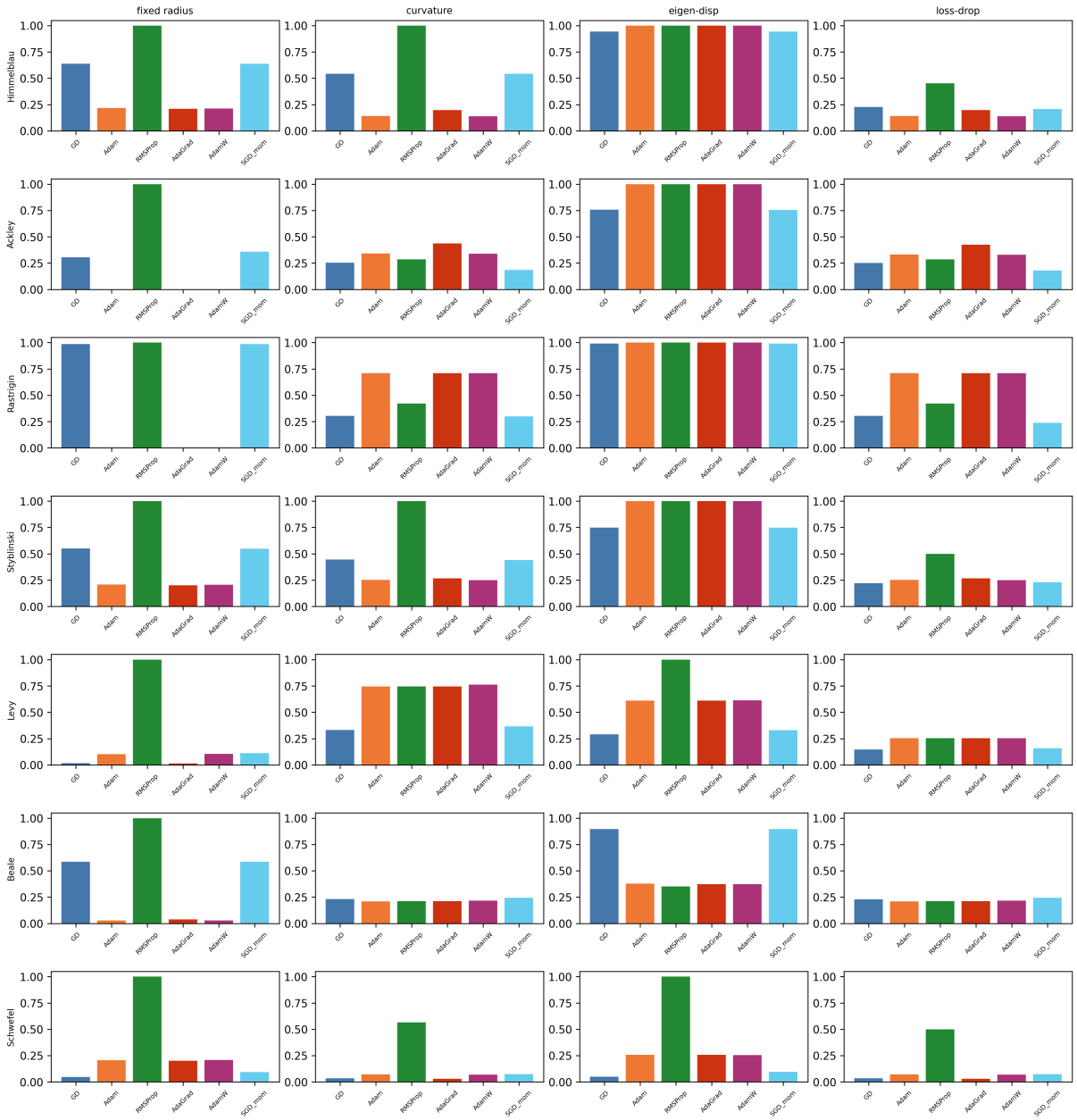


Figure 1: Best-LR SEE under all four criteria across seven functions.

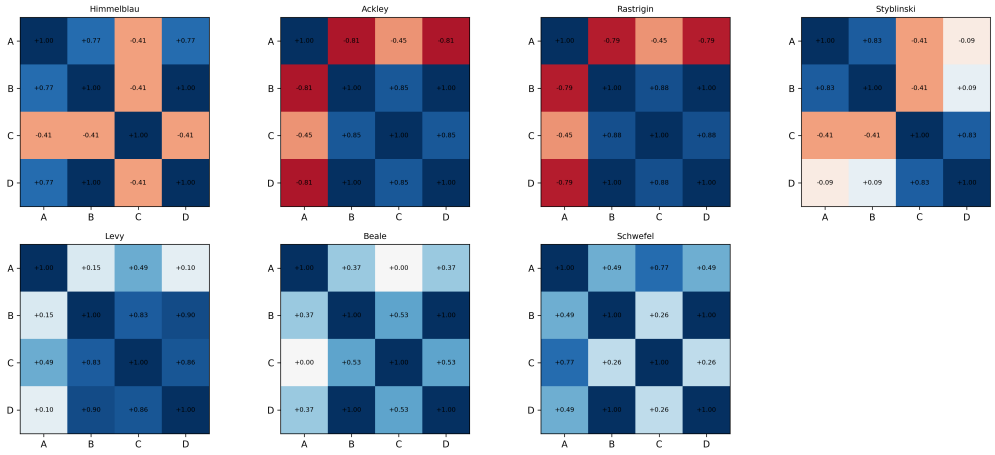


Figure 2: Pairwise Spearman ρ between criteria across all functions.

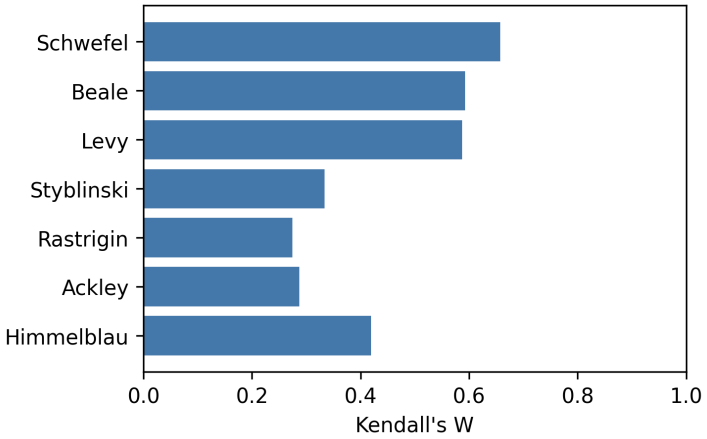


Figure 3: Kendall's W across criteria A, B, C, D per function.

5.4. Higher Dimensions

Rastrigin and Ackley are extended to $d = 10$ and $d = 50$. Only criteria A and B are evaluated. $N = 100$, $T_{\max} = 500$.

Table 5: $\rho(A, B)$ across dimensions. Rastrigin-50D is undefined because every optimiser achieves best-LR $\mathrm{SEE}_A = 1.000$.

	$d = 2$	$d = 10$	$d = 50$
Rastrigin	-0.79	-0.74	undefined
Ackley	-0.81	$+0.14$	$+0.78$

For Rastrigin the inversion persists at $d = 10$ and gets worse at $d = 50$: criterion A stops discriminating entirely, while criterion B still separates Adam/AdaGrad/AdamW (SEE

≈ 0.50) from GD/SGD+momentum (SEE < 0.08). For Ackley the inversion disappears because Ackley's $|\lambda_{\min}|$ drops from 17.11 at $d = 2$ to 0.70 at $d = 50$ as the cosine terms average out, moving the saddle into the flat regime where A and B agree.

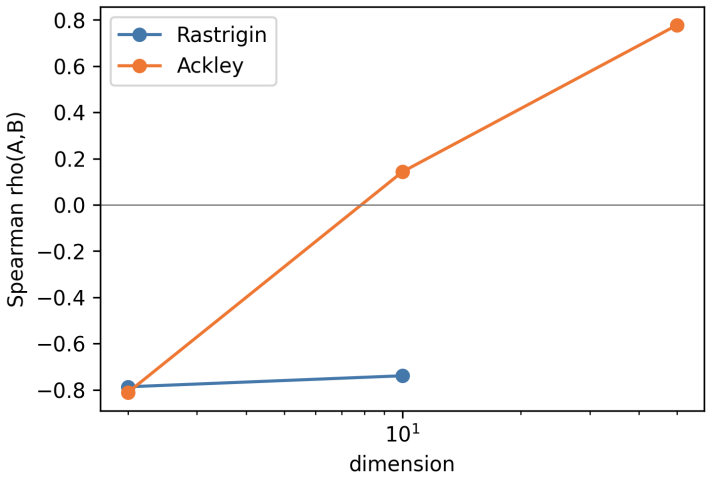


Figure 4: Spearman $\rho(A, B)$ vs dimension for Rastrigin and Ackley.

5.5. Neural Network Loss Surface

A small MLP (input 2, hidden 8, output 1, tanh, no hidden bias, $D = 25$ parameters) is trained on XOR replicated to 200 samples with Gaussian noise $\sigma = 0.05$. Saddles in parameter space are found using random initialisations plus `fsolve`. Two saddles are found. the one used has loss $= 0.173$, $\lambda_{\min} = -0.402$, $\lambda_{\max} = 7.646$. $N = 50$, $T_{\max} = 300$.

Table 6: SEE under criteria A and B on the XOR-MLP loss surface.

Optimiser	A	B
GD	0.000	0.000
Adam	0.091	0.010
RMSProp	1.000	0.006
AdaGrad	0.044	0.005
AdamW	0.092	0.010
SGD+mom	0.006	0.000

$\rho(A, B) = +0.81$, no inversion. This is consistent with the 2D results: the saddle found has $|\lambda_{\min}| = 0.402$, placing it in the flat-saddle regime where A and B tend to agree. SEE_B is near zero for all optimisers because this shallow saddle's Hessian takes a long time to become positive definite after the iterate moves away, while criterion A marks RMSProp as near-perfect because its large steps quickly move the parameter vector far from x_s.

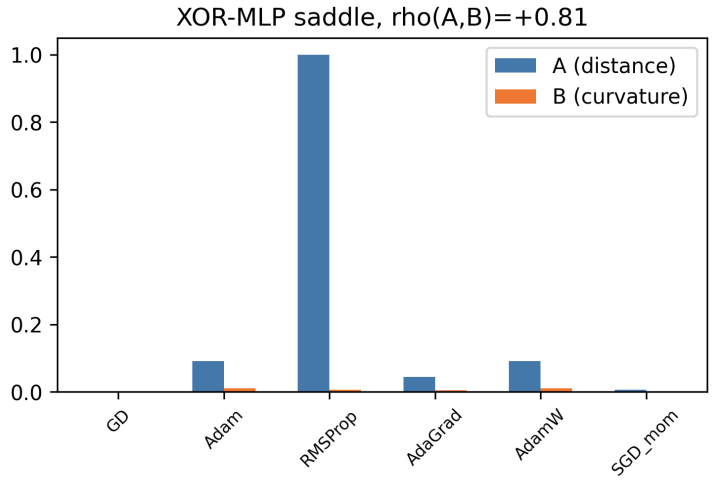


Figure 5: Criterion A vs B SEE on the XOR-MLP loss surface.

6. Limitations and Future Work

All 2D experiments are based on classical benchmark functions. Whether the rank inversion appears in the loss surfaces of deep networks remains to be shown, since finding saddle points with large $|\lambda_{\min}|$ in high-dimensional parameter spaces requires more specialized techniques than random initialization plus gradient root-finding.

The correlation $\rho = -0.70$ between the curvature and the sharpness is not significant at $n = 5$. A cleaner test would be a study with functions parameterised directly by the sharpness of curvature. Criteria B and D calculate the Hessian at each step, which is not feasible for large networks. Whether criterion B can be approximated with Lanczos on Hessian-vector products while preserving its discriminative properties is an open question.

7. Conclusion

In this paper we introduced Saddle Escape Efficiency (SEE) and showed that the choice of escape criterion can completely reverse optimiser rankings. Distance- and curvature-based criteria achieve Spearman rank correlations as low as $\rho = -0.81$ on standard benchmarks. Two criteria based on geometrical grounds agree with $\rho = +1.00$ on five of seven functions. In higher dimensions the distance criterion saturates completely whereas the curvature criterion still discriminates.

The inversion is not universal. It is present on sharp oscillating saddles and absent on flat ones. Criterion C discriminates on flat saddles and saturates on sharp ones. Together these results indicate that the choice of escape criterion determines not only the numbers but also the conclusions drawn from them. Any saddle-escape metric should specify and justify its criterion.

References

[1] Y. N. Dauphin, R. Pascanu, C. Gulcehre, K. Cho, S. Ganguli, and Y. Bengio, "Identifying and attacking the saddle point problem in high-dimensional non-convex optimization," in Proc. NIPS, 2014.

[2] J. Duchi, E. Hazan, and Y. Singer, "Adaptive subgradient methods for online learning and stochastic optimization," Journal of Machine Learning Research

[3] T. Tieleman and G. Hinton, "Lecture 6.5 RMSProp: Divide the gradient by a running average of its recent magnitude," COURSERA: Neural Networks for Machine Learning, 2012.

[4] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in Proc. ICLR, 2015.

[5] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in Proc. ICLR, 2019.

[6] M. Staib, S. Reddi, S. Kale, S. Kumar, and S. Sra, "Escaping saddle points with adaptive gradient methods," in Proc. ICML, 2019.

[7] K. Antonakopoulos, P. Mertikopoulos, G. Piliouras, and X. Wang, "AdaGrad avoids saddle points in non-convex optimization," in Proc. ICML, 2022.

[8] C. Jin, R. Ge, P. Netrapalli, S. M. Kakade, and M. I. Jordan, "How to escape saddle points efficiently," in Proc. ICML, 2017.

[9] M. Schmidt, "Machine learning convergence of gradient descent," Lecture Notes, University of British Columbia, 2017.

[10] S. Boyd and L. Vandenberghe, Convex Optimization. Cambridge University Press, 2004.

[11] B. Efron, "Bootstrap methods: Another look at the jackknife," Annals of Statistics

[12] R. Ge, F. Huang, C. Jin, and Y. Yuan, "Escaping from saddle points online stochastic gradient for tensor decomposition," in Proc. COLT, 2015.

[13] J. D. Lee, M. Simchowitz, M. I. Jordan, and B. Recht, "Gradient descent only converges to minimizers," in Proc. COLT, 2016.

[14] Y. Yuan, Deterministic Methods for Nonconvex Optimization. Springer, 2006.

[15] M. G. Kendall, Rank Correlation Methods. Charles Griffin, London, 1948.

[16] M. Ravaut and S. K. Gorti, "Gradient descent revisited via an adaptive online learning rate," arXiv:1801.09136, 2018.

[17] I. Goodfellow, O. Vinyals, and A. Saxe, "Qualitatively characterizing neural network optimization problems," in Proc. ICLR, 2015.

[18] H. Li, Z. Xu, G. Taylor, C. Studer, and T. Goldstein, "Visualizing the loss landscape of neural nets," in Proc. NeurIPS, 2018.

[19] N. S. Keskar, D. Mudigere, J. Nocedal, M. Smelyanskiy, and P. T. P. Tang, "On large-batch training for deep learning: Generalization gap and sharp minima," in Proc. ICLR, 2017.

[20] P. Foret, A. Kleiner, H. Mobahi, and B. Neyshabur, "Sharpness-aware minimization for efficiently improving generalization," in Proc. ICLR, 2021.

[21] A. C. Wilson, R. Roelofs, M. Stern, N. Srebro, and B. Recht, "The marginal value of adaptive gradient methods in machine learning," in Proc. NeurIPS, 2017.

[22] J. D. Lee, I. Panageas, G. Piliouras, M. Simchowitz, M. I. Jordan, and B. Recht, "First-order methods almost always avoid strict saddle points," Mathematical Programming